Python Notes (Lecture 8)

Object Oriented Programming

1. OOP in Python

To map code to **real-world scenarios**, we use objects. Programming with objects is called **object oriented programming (OOP)**.

For example, every student has a name and marks, and can do things like calculate an average. OOP lets us bundle that data and those actions together.

---

2. Class and Object

- A **class** is a blueprint for creating objects.
- An **object** (instance) is a real thing made from that blueprint.

```
# creating class
class Student:
    name = "karan kumar"

# creating object (instance)
s1 = Student()
print(s1.name)
```

Output:

```
karan kumar
```

---

3. Class and Instance Attributes

- **Class attribute** (`Class.attr`): shared by all objects of the class.
- **Instance attribute** (`obj.attr`): belongs to one particular object.

```python
class Student:
    college_name = "ABC College"      # class attribute

    def __init__(self, name):
        self.name = name              # instance attribute

s1 = Student("karan")
s2 = Student("arjun")

print(s1.name, s2.name)
print(Student.college_name)
print(s1.college_name)
```

Output:

```
karan arjun
ABC College
ABC College
```

If an object has an instance attribute with the same name as a class attribute, the instance attribute is used for that object.

---

4. __init__ Function (Constructor)

All classes have a function called `__init__()`, which is **always executed when an object is created**. It is used to give initial values to the object.

```python
# creating class
class Student:
    def __init__(self, fullname):
        self.name = fullname

# creating object
s1 = Student("karan")
print(s1.name)
```

Output:

```
karan
```

self is a reference to the current object. It is used to access the variables that belong to that object. Python passes it automatically, so we don't write it when creating the object.

5. Methods

Methods are **functions that belong to objects**. Their first parameter is `self`.

```
class Student:
    def __init__(self, fullname):
        self.name = fullname

    def hello(self):
        print("hello", self.name)

s1 = Student("karan")
s1.hello()
```

Output:

```
hello karan
```

Let's Practice

Q1. Create a Student class that takes name and marks of 3 subjects as arguments in the constructor. Then create a method to print the average.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def get_avg(self):
        avg = sum(self.marks) / len(self.marks)
        print(self.name, "average:", avg)

s1 = Student("Tony Stark", [99, 98, 97])
s1.get_avg()
```

Output:

```
Tony Stark average: 98.0
```

6. Static Methods

Static methods **don't use the self parameter**. They work at the class level and are marked with the `@staticmethod` decorator.

```
class Student:
    @staticmethod          # decorator
    def college():
        print("ABC College")

Student.college()
s1 = Student()
s1.college()
```

Output:

```
ABC College
ABC College
```

Decorators wrap another function to extend its behaviour without permanently changing it.

---

7. Important Concepts

Abstraction

Hiding the implementation details of a class and showing only the essential features to the user.

Example: when you call `car.start()`, you don't need to know how the engine works inside.

Encapsulation

Wrapping data and functions into a single unit (an object).

Example: the `Student` class keeps the name and marks together with the `get_avg()` method that works on them.

Let's Practice

Q2. Create an Account class with 2 attributes: balance and account number. Create methods for debit, credit and printing the balance.

```
class Account:
    def __init__(self, balance, acc_no):
        self.balance = balance
        self.acc_no = acc_no

    def debit(self, amount):
        self.balance -= amount
        print("Rs.", amount, "was debited")
        self.get_balance()

    def credit(self, amount):
        self.balance += amount
        print("Rs.", amount, "was credited")
        self.get_balance()

    def get_balance(self):
        print("Total balance =", self.balance)

acc1 = Account(10000, 12345)
acc1.debit(1000)
acc1.credit(500)
```

Output:

```
Rs. 1000 was debited
Total balance = 9000
Rs. 500 was credited
Total balance = 9500
```

A method can call another method of the same object using `self`, like `self.get_balance()`.